Use of Artificial Intelligence in automating coding – Pros and Cons

Tanushree Dhal[1], Sambit Gumansingh[2],Sanjana Basel[3]

[1] Student, B.Tech, Trident ,Bhubaneswar.

[2]Student, B.Tech, ASBM University, Bhubaneswar.

[3]Student, BCA,ASBM University, Bhubaneswar.

Abstract

Artificial intelligence (AI) has revolutionized the field of coding by automating complex programming tasks, enhancing code quality, and accelerating software development processes. AI-driven coding tools utilize machine learning models to generate, debug, and optimize code, enabling developers to focus on higher-level design and innovation. The integration of AI in coding promises a future where programming becomes more accessible, efficient, and adaptive to evolving technological demands. Emerging trends include the use of generative AI for real-time code synthesis, intelligent error detection, and personalized coding assistance. This evolution is expected to transform software engineering, fostering greater productivity, reducing human error, and enabling the creation of more sophisticated and reliable applications.

Introduction

Coding practices using AI tools have rapidly evolved, becoming an integral part of modern software development workflows. AI-powered coding assistants leverage advanced machine learning models to enhance productivity, reduce errors, and streamline the coding process (1, 6, 7, 13). These tools assist developers by generating code snippets, suggesting optimizations, detecting bugs, and even automating routine tasks, thereby enabling more efficient and accurate software creation (11, 17, 19).

Currently, AI coding tools are widely adopted across various programming environments and languages. They integrate seamlessly with popular integrated development environments (IDEs) and version control systems, offering real-time support and contextual recommendations (6, 18). The use of AI in coding not only accelerates development cycles but also facilitates learning and collaboration among developers by providing immediate feedback and code explanations (14, 19).

Examples of widely used AI coding tools include GitHub Copilot, which provides context-aware code completions and suggestions directly within code editors (7, 13); TabNine, which supports multiple languages and uses deep learning to predict code; and Kite, which offers intelligent completions and documentation lookup. These tools exemplify how AI enhances developer workflows by reducing repetitive tasks and accelerating coding processes (1, 6).

Despite these advancements, several major drawbacks remain a concern in the current trend. Out of several drawbacks, the major ones are:

1. AI-generated code can sometimes lack context awareness, leading to errors or inefficient implementations that require human oversight (6, 11).
2. Concerns regarding code security and privacy arise, especially when proprietary or sensitive codebases are involved (8, 10, 12).
3. The reliance on AI for coding risks diminishing developers' deep understanding of programming fundamentals, potentially impacting problem-solving skills and code maintainability (14, 16).
4. Current AI models may inherit biases from their training data, resulting in suboptimal or non-inclusive code patterns (9, 10).

To mitigate these challenges, several strategies can be employed. Enhancing AI models with improved context sensitivity and domain-specific knowledge can reduce errors and increase relevance (6, 11). Implementing strict data privacy protocols and secure environments when integrating AI tools will safeguard sensitive information (8, 12). Encouraging balanced use of AI assistance alongside continuous developer education ensures that foundational skills remain strong (14, 18). Additionally, diversifying training datasets and incorporating fairness-aware algorithms can minimize bias in code generation (9, 10). Addressing these challenges is essential to fully harness the potential of AI-driven coding while ensuring reliability, security, and ethical standards in future applications (1, 17, 20).

Challenges in Current Situation

In this paper we will mainly discuss on the above four challenges which needs to be addressed before coding can be automated to its maximum potential using AI.

AI-generated code can lack context awareness

AI-generated code can sometimes lack context awareness, which leads to errors or inefficient implementations requiring human oversight (6, 11, 17, 19). This limitation arises because AI coding tools typically generate code based on patterns learned from large datasets rather than a deep understanding of the specific application context, user intent, or broader system architecture (1, 6, 18). Below are detailed scenarios illustrating this issue, grounded in the context of AI coding assistants as described in the selected text.

Scenario 1: Misunderstanding Project-Specific Conventions and Constraints

An AI coding assistant like GitHub Copilot may generate syntactically correct code snippets but fail to align with project-specific coding conventions, architectural patterns, or domain-specific requirements (6, 12).

For example, in financial software requiring strict compliance and auditability, AI-generated code might omit required logging or encryption because these are not apparent in the immediate code context (10, 12).

This demonstrates the need for human oversight to ensure compliance with system standards.

Scenario 2: Inefficient Algorithm Selection Due to Missing Context

AI tools such as TabNine or Kite generate predictions without awareness of data size, performance constraints, or memory limitations (1, 6).For instance, the AI might propose a bubble sort instead of quicksort or mergesort because it is more common in the training data (16, 19).Humans must optimize and replace inefficient AI-generated implementations.

Scenario 3: Contextual Misinterpretation in Multi-Language or Framework Environments

In multi-framework projects, AI suggestions may be syntactically correct but clash with the overall architecture (6, 18).For example, React lifecycle methods suggested in a Node.js backend file illustrate context blindness (19).Such issues usually emerge during integration testing.

Scenario 4: Security Vulnerabilities from Context-Unaware Code Generation

AI assistants frequently fail to infer secure coding practices unless explicitly prompted (10, 12, 16).

For example, concatenating inputs directly into SQL queries may result in injection vulnerabilities (10).Human review is vital to prevent such security issues.

Scenario 5: Ignoring Legacy System Constraints

AI-generated code may use modern libraries or language features incompatible with legacy systems (6, 11).Such suggestions require developers to rewrite or adjust the code to meet older system requirements (20).

Scenario 6: Overfitting to Training Data Biases Without Contextual Adaptation

AI tools may reproduce common but outdated or insecure coding patterns from training datasets (9, 10, 16).For example, generating code with hardcoded credentials is a frequent risk (12).

Without context-aware filtering, these patterns compromise secure development.

Scenario 7: Misalignment with User Intent in Ambiguous Requests

When user prompts are vague, AI assistants generate generic but unsuitable implementations based on probability rather than intent (17, 19).This requires iterative human refinement and validation.These scenarios demonstrate that despite major productivity benefits, AI coding tools still lack deep context awareness, often leading to errors, inefficiencies, and security vulnerabilities (1, 6, 11, 12, 17).Human expertise remains essential to ensure code quality, maintainability, and compliance with organizational standards (14, 18, 20).

Code Security and Privacy

Concerns regarding code security and privacy in automating coding using AI are critical, especially when proprietary or sensitive codebases are involved (8, 10, 12).These concerns arise from AI model training, cloud-based processing, and potential exposure of sensitive information.

Scenario 1: Exposure of Proprietary Code Through Cloud-Based AI Assistants

Cloud-based tools like GitHub Copilot or TabNine send code snippets to external servers (7, 8).

Risk: possible unauthorized access or data retention without user control (12).

Implication: proprietary algorithms or logic may unintentionally leak. a company developing a proprietary financial algorithm may have its core logic partially sent to an AI service. If the AI provider logs or retains this data for model improvement without adequate anonymization, competitors might gain access indirectly. Even if the provider does not intend to misuse the data, inadvertent breaches or insider threats could expose sensitive information.

Scenario 2: AI Training Data Containing Proprietary Code

AI models trained on large datasets may unintentionally reproduce proprietary or copyrighted code (7, 13, 16).This poses intellectual property and security risks.

Concern: If the training data includes proprietary or confidential code, AI-generated suggestions might reproduce this code verbatim or reveal patterns that compromise uniqueness or security. This leakage can result in unintentional sharing of sensitive code fragments with unauthorized parties.

Implication: A developer using an AI assistant might receive code completions that closely resemble proprietary implementations from another company. This not only risks intellectual property infringement but could also introduce vulnerabilities if the original code contained security flaws

Scenario 3: Lack of Context Awareness Leading to Security Vulnerabilities

AI tools may generate insecure code such as weak encryption or unsanitized input handling (10,12).This can result in compliance violations (e.g., GDPR, HIPAA) (9, 15).

Concern: AI-generated code may introduce security vulnerabilities such as improper input validation, insecure data handling, or weak encryption schemes, especially when dealing with sensitive data. Without human oversight, these vulnerabilities can propagate into production systems.

Implication: For instance, an AI assistant might suggest code that stores sensitive user credentials in plaintext or fails to sanitize inputs, leading to potential data breaches or compliance violations (e.g., GDPR, HIPAA). This scenario is particularly risky when the codebase handles personal or financial information.

Scenario 4: Insider Threats and Misuse of AI Coding Tools

AI assistants can be misused by insiders to reproduce or exfiltrate sensitive modules (8, 12).

Lack of monitoring increases risk.

Concern: Malicious insiders could leverage AI tools to exfiltrate proprietary algorithms or sensitive code by querying the AI for specific implementations or generating code that mimics restricted components.

Implication: This risk is heightened in environments where AI tools are integrated without strict access controls or monitoring. It could lead to intellectual property theft, sabotage, or unauthorized distribution of confidential code.

Scenario 5: Compliance and Data Privacy Regulations

AI tools may conflict with regulations if they transfer or store sensitive information improperly (9, 12, 15).This is especially critical in healthcare, banking, or government sectors.

Scenario 6: Dependency on Third-Party AI Providers and Vendor Risks

Organizations must trust external AI vendors for secure handling of code (8, 10).

Policy changes or breaches may expose sensitive data.

> Concern: AI tools that do not guarantee compliance with data residency, encryption standards, or audit requirements pose legal and operational risks. The use of such tools could lead to violations of laws like GDPR, CCPA, or industry-specific mandates.
>
> Implication: A healthcare provider using AI coding assistants might inadvertently expose patient data embedded in code comments or configurations. Failure to control data flows and ensure compliance can result in hefty fines, reputational damage, and operational disruptions.

Mitigation Strategies: To address these security and privacy concerns, organizations should adopt multiple strategies:(consistent with references 1, 8, 10, 12, 17, 20)

1.On-Premises or Private Cloud Deployment: Using AI coding tools deployed within the organization's secure infrastructure minimizes data exposure to external servers

2. Data Anonymization and Encryption: Ensuring that any code or data sent to AI services is anonymized and encrypted reduces the risk of leakage.

3. Access Controls and Monitoring: Implementing strict user permissions and monitoring AI tool usage helps detect and prevent insider threats or misuse.

4. Human Oversight and Code Review: Maintaining rigorous code review processes ensures AI-generated code adheres to security policies and best practices.

5. Vendor Due Diligence: AI providers for compliance certifications, security posture, and transparent data handling policies is essential.

6. Training and Developer Awareness: Educating developers on the risks and responsible use of AI coding assistants promotes a security-conscious culture.

These scenarios and mitigation approaches highlight the complexity and importance of addressing code security and privacy when automating coding using AI. Organizations must balance the productivity gains of AI tools with robust safeguards to protect proprietary and sensitive codebases.

Impact on the developer's potential

AI-generated code can sometimes lack context awareness, which leads to errors or inefficient implementations requiring human oversight (6, 11, 17, 19). This limitation arises because AI coding tools typically generate code based on patterns learned from large datasets rather than a deep understanding of the specific application context, user intent, or broader system architecture (1, 6, 18). Below are detailed scenarios illustrating this issue, grounded in the context of AI coding assistants as described in the selected text.

Scenario 1: Misunderstanding Project-Specific Conventions and Constraints

An AI coding assistant like GitHub Copilot may generate syntactically correct code snippets but fail to align with project-specific coding conventions, architectural patterns, or domain-specific requirements (6, 12).For example, in financial software requiring strict compliance and auditability, AI-generated code might omit required logging or encryption because these are not apparent in the immediate code context (10, 12).

This demonstrates the need for human oversight to ensure compliance with system standards.

Scenario 2: Inefficient Algorithm Selection Due to Missing Context

AI tools such as TabNine or Kite generate predictions without awareness of data size, performance constraints, or memory limitations (1, 6).

For instance, the AI might propose a bubble sort instead of quicksort or mergesort because it is more common in the training data (16, 19).

Humans must optimize and replace inefficient AI-generated implementations.

Scenario 3: Contextual Misinterpretation in Multi-Language or Framework Environments

In multi-framework projects, AI suggestions may be syntactically correct but clash with the overall architecture (6, 18).

For example, React lifecycle methods suggested in a Node.js backend file illustrate context blindness (19).Such issues usually emerge during integration testing.

Scenario 4: Security Vulnerabilities from Context-Unaware Code Generation

AI assistants frequently fail to infer secure coding practices unless explicitly prompted (10, 12, 16).For example, concatenating inputs directly into SQL queries may result in injection vulnerabilities (10).Human review is vital to prevent such security issues.

Scenario 5: Ignoring Legacy System Constraints

AI-generated code may use modern libraries or language features incompatible with legacy systems (6, 11).Such suggestions require developers to rewrite or adjust the code to meet older system requirements (20).

Scenario 6: Overfitting to Training Data Biases Without Contextual Adaptation

AI tools may reproduce common but outdated or insecure coding patterns from training datasets (9, 10, 16).For example, generating code with hardcoded credentials is a frequent risk (12).

Without context-aware filtering, these patterns compromise secure development.

Scenario 7: Misalignment with User Intent in Ambiguous Requests

When user prompts are vague, AI assistants generate generic but unsuitable implementations based on probability rather than intent (17, 19).

This requires iterative human refinement and validation.

These scenarios demonstrate that despite major productivity benefits, AI coding tools still lack deep context awareness, often leading to errors, inefficiencies, and security vulnerabilities (1, 6, 11, 12, 17).Human expertise remains essential to ensure code quality, maintainability, and compliance with organizational standards (14, 18, 20).

Code Security and Privacy

Concerns regarding code security and privacy in automating coding using AI are critical, especially when proprietary or sensitive codebases are involved (8, 10, 12).

These concerns arise from AI model training, cloud-based processing, and potential exposure of sensitive information.

Scenario 1: Exposure of Proprietary Code Through Cloud-Based AI Assistants

Cloud-based tools like GitHub Copilot or TabNine send code snippets to external servers (7, 8).

Risk: possible unauthorized access or data retention without user control (12).

Implication: proprietary algorithms or logic may unintentionally leak.

Scenario 2: AI Training Data Containing Proprietary Code

AI models trained on large datasets may unintentionally reproduce proprietary or copyrighted code (7, 13, 16).This poses intellectual property and security risks.

Scenario 3: Lack of Context Awareness Leading to Security Vulnerabilities

AI tools may generate insecure code such as weak encryption or unsanitized input handling (10, 12).This can result in compliance violations (e.g., GDPR, HIPAA) (9, 15).

Scenario 4: Insider Threats and Misuse of AI Coding Tools

AI assistants can be misused by insiders to reproduce or exfiltrate sensitive modules (8, 12).

Lack of monitoring increases risk.

Scenario 5: Compliance and Data Privacy Regulations

AI tools may conflict with regulations if they transfer or store sensitive information improperly (9, 12, 15).This is especially critical in healthcare, banking, or government sectors.

Scenario 6: Dependency on Third-Party AI Providers and Vendor Risks

Organizations must trust external AI vendors for secure handling of code (8, 10).

Policy changes or breaches may expose sensitive data.

Mitigation Strategies

(consistent with references 1, 8, 10, 12, 17, 20)

1. On-Premises or Private Cloud Deployment
2. Data Anonymization and Encryption
3. Access Controls and Monitoring
4. Human Oversight and Code Review
5. Vendor Due Diligence
6. Training and Developer Awareness

Conclusion

Automating coding using AI represents a significant advancement in software development, offering substantial improvements in productivity, code quality, and innovation. AI-powered coding assistants streamline the development process by generating, completing, and debugging code across various programming languages, thus accelerating workflows and reducing manual effort. These tools, integrated into popular development environments, provide real-time suggestions and automate routine tasks, benefiting both novice and experienced developers. However, despite these advantages, challenges remain that must be addressed to ensure the responsible and effective use of AI in coding. Limitations such as insufficient context awareness can lead to errors requiring human intervention, while concerns about code security and privacy highlight the need for stringent data protection measures. Additionally, overreliance on AI may weaken developers' foundational skills, impacting long-term code maintainability and problem-solving capabilities. Biases inherited from training data also present risks of generating suboptimal or non-inclusive code patterns. To fully realize the potential of AI-driven coding, it is essential to enhance AI models with better context sensitivity, enforce robust privacy protocols,

and promote balanced use alongside ongoing developer education. Incorporating diverse training datasets and fairness-aware algorithms will further improve code quality and inclusivity. Addressing these challenges will enable AI automation to support reliable, secure, and ethical software development, ultimately transforming coding practices while preserving the critical role of human expertise.

References

1. (2024). Generative Ai in Software Development : an Overview and Evaluation of Modern Coding Tools. *International Journal For Multidisciplinary Research*, *6*(3). https://doi.org/10.36948/ijfmr.2024.v06i03.23271
2. Adetayo, A. J., Sanni, B. A., & Aborisade, M. O. (2024). Microsoft Copilot and Anthropic Claude AI in education and library service. *Library Hi Tech News*. https://doi.org/10.1108/lhtn-01-2024-0002
3. Dhoni, P. S. (2024). From Data to Decisions: Enhancing Retail with AI and Machine Learning. *International Journal of Computing and Engineering*, *5*(1), 38–51. https://doi.org/10.47941/ijce.1660
4. Alsamhori, A. R. F., Alsamhori, J. F., & Alsamhori, A. F. (2023). ChatGPT Role in a Medical Survey. *High Yield Medical Reviews*, *1*(2). https://doi.org/10.59707/hymrtffp5435
5. Cao, K. (2024). Predicting the behavior intention of intelligent office software. *SHS Web of Conferences*, *181*, 01009. https://doi.org/10.1051/shsconf/202418101009
6. Corso, V., Micucci, D., Mariani, L., & Riganelli, O. (2024). *Generating Java Methods: An Empirical Assessment of Four AI-Based Code Assistants*. *10*, 13–23. https://doi.org/10.1145/3643916.3644402
7. Bird, C., Zimmermann, T., Ford, D., Forsgren, N., Lowdermilk, T., Kalliamvakou, E., & Gazit, I. (2022). Taking Flight with Copilot. *Queue*, *20*(6), 35–57. https://doi.org/10.1145/3582083
8. Ismail, I., Kurnia, R., Widyatama, F., Wibawa, I. M., Brata, Z. A., Ukasyah, U., Nelistiani, G. A., & Kim, H. (2025). Enhancing Security Operations Center: Wazuh Security Event Response with Retrieval-Augmented-Generation-Driven Copilot. *Sensors (Basel, Switzerland)*, *25*(3), 870. https://doi.org/10.3390/s25030870
9. Gupta, A., & Mishra, M. (2022). Ethical Concerns While Using Artificial Intelligence in Recruitment of Employees. *Business Ethics and Leadership*, *6*(2), 6–11. https://doi.org/10.21272/bel.6(2).6-11.2022
10. Sai, S., Yashvardhan, U., Chamola, V., & Sikdar, B. (2024). Generative AI for Cyber Security: Analyzing the Potential of ChatGPT, DALL-E, and Other Models for

Enhancing the Security Space. *IEEE Access*, *12*, 53497–53516. https://doi.org/10.1109/access.2024.3385107

11. Ferdowsi, K., Huang, R. (Lisa), Polikarpova, N., Lerner, S., & James, M. B. (2024). *Validating AI-Generated Code with Live Programming*. 1–8. https://doi.org/10.1145/3613904.3642495
12. Klemmer, J. H., Horstmann, S. A., Votipka, D., Rashid, A., Patnaik, N., Burton, C., Fahl, S., Ludden, C., Rahman, A., Powers, C., Naiakshina, A., Lipford, H. R., & Massacci, F. (2024). *Using AI Assistants in Software Development: A Qualitative Study on Security Practices and Concerns*. 2726–2740. https://doi.org/10.1145/3658644.3690283
13. Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). *The Impact of AI on Developer Productivity: Evidence from GitHub Copilot*. Cornell University. https://doi.org/10.48550/arxiv.2302.06590
14. Güner, H., & Er, E. (2025). AI in the classroom: Exploring students' interaction with ChatGPT in programming learning. *Education and Information Technologies*, *30*(9), 12681–12707. https://doi.org/10.1007/s10639-025-13337-7
15. Venkatesh, K. P., Raza, M. M., & Kvedar, J. C. (2023). Automating the overburdened clinical coding system: challenges and next steps. *NPJ Digital Medicine*, *6*(1). https://doi.org/10.1038/s41746-023-00768-0
16. Al Madi, N. (2022). *How Readable is Model-generated Code? Examining Readability and Visual Inspection of GitHub Copilot*. 1–5. https://doi.org/10.1145/3551349.3560438
17. Mozannar, H., Bansal, G., Horvitz, E., & Fourney, A. (2024). When to Show a Suggestion? Integrating Human Feedback in AI-Assisted Programming. *Proceedings of the AAAI Conference on Artificial Intelligence*, *38*(9), 10137–10144. https://doi.org/10.1609/aaai.v38i9.28878
18. Corso, V., Riganelli, O., Micucci, D., & Mariani, L. (2024). *Assessing AI-Based Code Assistants in Method Generation Tasks*. 380–381. https://doi.org/10.1145/3639478.3643122
19. Liang, J. T., Yang, C., & Myers, B. A. (2024). *A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges*. 1–13. https://doi.org/10.1145/3597503.3608128

20. Lyu, M. R., Tan, S. H., Thongtanunam, P., Ray, B., & Roychoudhury, A. (2025). Automatic Programming: Large Language Models and Beyond. *ACM Transactions on Software Engineering and Methodology*, *34*(5), 1–33. https://doi.org/10.1145/3708519.